

识途 · 演讲稿（三人分段）

按三人分工分段，从实现讲起 | 合计约 7 分钟 | 灰色小字是提示与交接点，不用念

第一段 | 我：Agent 编排与规划生成 约 2 分 30 秒

（上台，接住话筒即开始）

大家好，我们是识途团队。我用一句话交代项目定位，然后直接讲实现。

识途是一个自由行的旅行规划 Agent：用户用自然语言说需求，系统在右侧工作台上一步步比较景点、住宿房型、往返班次和餐饮，最后生成一份带来源、标明待确认事项的旅行计划书。

我先用一句话演示。我输入：十月二十号从上海出发去青岛玩三天，两个人，预算五千，喜欢美食和人文。

（输入这句话，等右侧出现景点候选）

它没有直接给我一份攻略，而是先把我的话拆成条件，再逐个去查真实数据。右边这四个景点就是这次查到的候选，每个都带介绍和来源。

我点开一个看详情。有介绍、有营业资料、有电话和人均，也有地图位置。请注意这一行——门票和预约状态写的是“待核实”。这不是偷懒：门票价格和预约名额本来就会变，查不到我们宁可写清楚，也不写一个看起来很确定的数字。

我这段负责的是这套流程的编排，也就是用户那句话到一串动作之间发生了什么。

第一步是意图识别。我们用 DeepSeek 的函数调用把自然语言变成结构化动作，比如“去查青岛的景点，关键词是人文和美食”。第二步是编排，用 LangGraph 搭了一条 understand 到 execute 的主链，执行节点按动作去调用查询、选择或者规划模块。第

三步是状态，用户每一次选择都会落库并生成一个新版本，所以改条件、撤销、重新规划都有据可依。

这里有一条我们自己定的规矩，我觉得是这个项目最关键的设计：模型只负责判断和表达，所有数字必须来自接口。模型可以决定查什么、怎么排序、怎么措辞，但它不能编时刻、编价格、编距离。所以模型输出之后，程序还会再校验一遍——景点必须是用户选过的、每个只能出现一次、日期必须在这次旅行范围内、每段游玩时长也有上下限。不合法就带着具体错误让模型改一次，第二次还不行就明确报错，绝不静默编一份行程出来。

长行程也一样：我们支持到六十个游玩日，模型每轮最多处理十六个地点，逐段限定日期和 ID，并核对有没有遗漏和重复。

我这段就到这里，数据部分交给成员二。

第二段 | 成员二：多源数据与工具服务 约 2 分 30 秒

(接过话筒，从演示接着往下走)

我接着往下演示。景点选好三个、点完成之后，系统才进入住宿，因为住宿要按我选的景点位置来比较。

(点三个景点，点"完成景点选择")

酒店和房型来自途牛。展开房型能看到具体报价、含不含早餐、怎么退改。这里有一个口径要说清楚：列表上那个价格是起价，不等于最后的总价，界面上我们写明了。住宿确认之后，它自动整理了往返班次，去程推了一趟高铁，我也可以改成飞机——同样要说清楚，余票是查询那一刻的快照，选了不等于订到了。

最后生成计划书。这份计划书里的交通时间、缓冲、以及地图上这条红色实线，都是真实查询回来的；查不到的时候我们只画顺序示意，绝不当成导航。

我这段负责的是这些数据从哪来、怎么变成能用的。

我们接了三个数据源：景点和路线来自高德，酒店房型、往返班次和门票来自途牛的 MCP 服务，天气来自和风。技术上，我们自己写了一个本地的 MCP 工具服务，把地点查询、路线计算、天气和本地资料检索封装成工具；途牛那边走的是 Streamable HTTP 的 MCP 接入，并且只开放四个只读查询工具白名单。

数据进来之后有三件事必须做。第一是统一：三家的字段名、单位、价格口径都不一样，我们收敛成一套结构，而且每个候选都带上是哪来的、什么时候查的，界面上点开就能看到来源。第二是限额：途牛的接口是免费额度，官方限制是每分钟五次、每天五十次，而且所有 key 共享。所以我们给项目自己设了每日查询预算，还加了最小请求间隔。这也是为什么有的操作会等十几秒——不是程序慢，是我们在按接口的规矩排队。第三是异常：查不到、没票、没有房、名称匹配不上，这些我们分开处理。没票的班次后端直接不允许选入；房型匹配不上就标成待核实、并且不计算路线；单一路线查失败就降级成一句可读提示，不让它把整个流程卡死。

我这段就到这里，检索和校验交给成员三。

第三段 | 成员三：知识检索与规划校验 约 2 分钟

(接过话筒)

我这段讲两件事：判断依据从哪来，以及我们怎么保证计划书里的时间是真能走通的。

先说依据。计划书里的介绍和提示不能是模型编的，所以我们整理了一份官方资料库，目前三十三条，包括景区资料和四个城市的目的地概览。检索用的是字符级的 TF-

IDF 加关键词重排。这里要说明一下，我们没有上向量数据库——这个量级的资料用字符特征就够了，零外部依赖；等真实检索的失败率证明它不够用，再换也不迟，这个判断我们写在代码注释里。每条资料都保留来源地址、采集时间和适用范围；如果是网页快照，我们会明确标注适用日期需要核实，不会把它当成出游当天的状态。

再说校验，这是我觉得最容易被做成“走过场”的部分，所以我们全交给程序做。日程排出来之后，会过几层检查：时间有没有重叠、当天到底排不排得下、到达那天有没有留出九十分钟做入境和接驳、返程前有没有留出两小时、以及我们选的这一餐在那个时段到底来不来得及吃。这些判否不依赖模型自觉。

还有一个细节我想强调：时间轴和选餐用的是同一套窗口计算。也就是说，如果某一餐在时间上排不进去，用户在选的那一刻就会被拦住并看到原因，而不是选完之后在计划书里被悄悄丢掉。这个口径是我们专门收敛到一个模块里的，因为我们踩过坑——口径不一致就会出现“选得上、排不进”。

最后，计划书还会让一个独立的审核模型再看一遍，它专门挑“是不是把建议说成了事实”“有没有时间或位置风险”，最多列六条问题。它只是第二意见，不是唯一验收者。

目前这套东西有 143 项后端测试和三套浏览器场景检查，全部通过。

三个人讲完了。识途现在能跑通需求、景点、住宿房型、往返交通、餐饮到计划书的完整链路。它最大的特点不是能聊，而是每一个事实性的字段都能说清出处，每一处没核实的都敢写“待核实”。

我们下一步想做三件事：把改一餐、改一晚的局部重规划做成不用重跑全流程；把预算从现在的住宿参考扩到分类合计；还有把体验包的接口额度按使用人隔离，让更多人可以各自独立体验。

我的介绍到这里，谢谢各位。

附：分段与上台前检查（不念）

分段	时长	内容
第一段 我	约 2 分 30 秒	项目一句话定位 + 演示前半 (景点、详情) + Agent 编排与 程序校验
第二段 成员二	约 2 分 30 秒	演示后半 (住宿、班次、计划 书) + 三源接入、字段统一、限 额与异常处理
第三段 成员三	约 2 分钟	知识检索与依据 + 时间与餐次 校验 + 独立审核 + 数据指标 + 收尾

- 启动服务：双击 启动.cmd，或运行 .\启动.ps1，浏览器打开 <http://127.0.0.1:8767>。
- 演示账号提前注册好，并确认“我的旅行”里有一趟已完成的示例旅行作为兜底。
- 演示前先跑一次 `pytest tests -q` 确认 143 passed，避免现场等测试。
- 途牛接口每分钟 5 次、每天 50 次共享；演示前不要反复查询，关键的住宿与班次建议提前查一次留缓存。
- 讲“待核实”“余票是快照”“选定不等于预订”这三处时放慢语速，这是最能建立可信度的地方。
- 如果现场网络不稳，直接打开已生成的计划书讲来源与待确认事项，不要现场重查。
- 交接点：第一段结尾“数据部分交给成员二”、第二段结尾“检索和校验交给成员三”，建议提前对一遍手势。